DATAML200 DEEP LEARNING · 2026–2027 · EXERCISE 3

Exploring hyper-parameters

Instructions for students. The notebook `ex3.ipynb` has the full details; if this handout and the notebook disagree, the notebook is right.

What you do

In this exercise you train a small neural network to recognise **environmental sounds**: a dog barking, rain, a crying baby, a chainsaw, and 46 other classes. The network is simple. The main work is to **explore its hyper-parameters**: you train many models, read plots of their losses, decide which settings work best, and finally commit to your own best configuration.

The audio clips are turned into **log-mel spectrograms**, small images of how the energy at each frequency changes over time. One model then trains in a few seconds on a laptop CPU, so you can afford to train dozens of them.

Getting started

You get `Ex3_train_deep_NN.zip` with three files: the notebook `ex3.ipynb`, `ex3_helpers.py` and these instructions, `ex3_instructions.pdf`.

Before you start: install librosa, the audio library used for the features, if you do not have it: `pip install librosa`. Keep the file `ex3_helpers.py` next to the notebook. It holds the code that is not part of your tasks: downloading the dataset, the plots, and the code behind the tests. You do not need to read it.

Run time: all training in this notebook takes about 10–15 minutes on a laptop CPU. In addition, the dataset (650 MB) is downloaded once and its features are computed once (about a minute). A GPU does not help here: the models are so small that a CPU is about as fast.

Dataset. ESC-50 is a collection of 2000 environmental sound clips of 5 seconds, 40 clips for each of 50 classes: animals, natural sounds such as rain and wind, human sounds such as coughing and snoring, domestic sounds, and urban sounds such as sirens and engines.

folds	clips	used for
1, 2, 3	1200	training
4	400	**validation**: comparing hyper-parameters
5	400	**test**: looked at only once, at the very end

Tasks and points

task	what you do	points
1. Log-mel features	Implement `log_mel(waveform, sr)` with librosa: the features of every clip.	2
2. The network	Build `SoundNet`, a 1-D CNN: blocks of Conv1d → BatchNorm1d → ReLU → MaxPool1d, then an average over time, dropout and a linear layer to the 50 classes.	4
3. The training and evaluation loops	Write `train_one_epoch` and `evaluate`.	4
4. One sweep function for every hyper-parameter	Write `sweep(base_config, grid, seeds)`: train every combination of the hyper-parameter values in `grid`, with every seed, and average the final results over the seeds.	4
5. Exploration sessions	Run six exploration sessions, update `my_config` after each one, and answer three multiple-choice questions.	3
6. Your final configuration	Choose your final configuration. The notebook trains it and saves `best_model.pth`, which is graded by its final validation loss.	3
	total	**20**

For tasks 1–4, the visible tests give half of the points and hidden tests, run after submission, the other half. The 3 questions of Task 5 give 1 point each.

The exploration sessions (Task 5)

Now you decide. You keep your current best settings in `my_config`. Each session sweeps one or two hyper-parameters around `my_config` with `run_sweep(sweep, my_config, grid, title)`. It calls your `sweep`, shows its progress and a plot that grows after every training run, and at the end plots the results your `sweep` returned. You then **update `my_config` with the values you choose** and go on to the next session. This is how hyper-parameters are often tuned by hand: one or two at a time, keeping the best values found so far.

session	values tried	runs
1. Learning rate	`lr`: 1e-4, 3e-4, 1e-3, 3e-3, 1e-2, 3e-2, 1e-1	7
2. Batch size and learning rate	`batch_size`: 16, 32, 64, 128 `lr`: 3e-4, 1e-3, 3e-3, 1e-2	16
3. Number of blocks and channels	`n_layers`: 1, 2, 3, 4, 5 `channels`: 16, 32, 64, 128, 256	25
4. Kernel size	`kernel_size`: 1, 3, 5, 7, 9	5
5. Dropout and training length	`dropout`: 0.0, 0.2, 0.4, 0.6 `epochs`: 20, 40, 80	12
6. Learning rate, again	`lr`: 3e-4, 1e-3, 3e-3, 1e-2 seeds 0, 1	8

Do not just copy the star. Runs with different seeds differ by about 0.05 in validation loss. If two values are within that noise of each other, the plot cannot tell which is better: prefer the cheaper or simpler one, or run the sweep with more seeds, e.g. `run_sweep(..., seeds=(0, 1))`.

Your own sweeps (optional). After the six sessions you can explore as many hyper-parameters and combinations as you like. With three or more hyper-parameters, the plot shows slices of the grid. `plot=False` turns the plots off, and `csv_file='my_runs.csv'` saves every run in a CSV file as soon as it is done.

Your final model (Task 6)

Your saved model is graded by its **final validation loss**:

final validation loss	points
at most 2.2	1
at most 1.9	2
at most 1.7	3

For comparison, the starting configuration has a validation loss of about 3.7, and fixing only its learning rate gives about 2.4. Your configuration must stay within these limits: `n_layers` 1–5, `channels` at most 256, `kernel_size` at most 9, `batch_size` at least 8, `epochs` at most 100.

What to submit

Deliverables: submit the notebook `ex3.ipynb` and your final model `best_model.pth`, which the notebook saves in Task 6. Do not rename the notebook.

Note: the variable `feedback_txt` is used for autograding. Do not modify or delete it. You may change `skip_training` in the cell where it is defined, but nowhere else. Set it to `True` to run the tests without training.